Cryptographically Enforced Delegated Spending Protocol

Anjul kc

November 2026

Abstract

This work proposes the concept of a Delegated Spending Protocol (DSP), a cryptographic approach to policy-controlled spending that does not rely on account-based delegation. Instead of identifying a spender through a blockchain address, the protocol uses delegated secret keys and proof of possession to authorize spending operations. It aims to support incremental consumption of allowances while enforcing predefined spending policies and tracking the remaining resources on-chain. By separating transaction authorization from transaction submission, the protocol also explores a way to let users interact with blockchain applications without directly managing wallets or paying transaction fees themselves. The proposed design uses proxy-signature-based authorization to keep on-chain verification lightweight, without requiring pairing-based operations or trusted setup.

The initial version provides a High level idea, abstracting the architecture, cryptography and security assumption.

Introduction

Delegation, in its simplest form, is the act of granting another party authority to perform certain operations under a policy defined by the delegator. One of the elegant applications of cryptography in this domain is the *Proxy Signature*, which allows an original signer (the delegator) to grant another party (the proxy signer) the ability to sign messages on their behalf.

Depending on the construction, delegation can be achieved through several approaches:

1. **Private-key disclosure:** The delegator reveals their entire private key to the proxy signer, effectively transferring unrestricted signing authority.
2. **Private-key derivation:** The delegator uses their private key to derive a separate delegation key, which is then provided to the proxy signer.
3. **Policy-based delegation:** The delegator grants signing authority subject to predefined restrictions or policies.

The latter two approaches are particularly powerful because they allow the delegator to retain control over their original private key while restricting the authority granted to the proxy signer. Depending on the construction, these restrictions may include limitations on the messages that can be signed, the scope of authority, or the duration of delegation. Furthermore, revocation mechanisms can be incorporated to invalidate previously granted delegation rights.

Motivation

Delegated spending and allowance mechanisms are well-established concepts in blockchain applications. They allow an asset owner to authorize another party to spend or manage a predefined portion of their resources under specified conditions. Although several implementations of this concept already exist, many commonly adopted approaches rely on account-based authorization. In such systems, the owner explicitly identifies a spender through a blockchain address and grants that address permission to consume resources according to a predefined allowance or policy. Consequently, the authorization mechanism remains closely tied to the identity of a blockchain account.

Solana's Spend Permissions is an example of this account-oriented approach. Other implementations explore more cryptographic forms of delegation. For instance, OpenZeppelin's confidential token mechanisms explore zero-knowledge proofs for operations involving delegated spending authority, including proving possession of the required cryptographic credentials without directly revealing them. Such approaches demonstrate the potential of cryptographic delegation, although they may introduce additional computational and implementation complexity.

This protocol explores an alternative direction by using proxy-signature-based delegation. Rather than identifying the spender through a blockchain account, the protocol associates spending authority with a delegated secret key. A consumer demonstrates possession of this key by producing a valid authorization proof, allowing the contract to verify the operation independently of the consumer's blockchain account identity.

This distinction also creates an opportunity to separate transaction authorization from transaction submission. A consumer does not necessarily need to own or operate a blockchain wallet, maintain a funded account, or interact directly with a wallet interface. A middleware or relayer can submit the authorized transaction on their behalf, while the contract independently verifies the cryptographic proof and enforces the associated spending policy.

The proposed construction aims to achieve this without introducing a zero-knowledge circuit, trusted setup, or pairing-based cryptographic operations. Instead, it relies on comparatively lightweight elliptic-curve operations and hash functions for authorization and verification. This design direction seeks to reduce the computational requirements of on-chain verification while retaining cryptographically enforced delegation.

Beyond the authorization mechanism itself, the protocol focuses on partial consumption of delegated allowances. A delegated authority can be exercised incrementally, with the contract tracking the remaining allowance and enforcing the associated policy throughout its lifetime. This makes the construction relevant to applications such as controlled payments, recurring allowances, subscriptions, and other scenarios where spending authority should remain valid across multiple transactions rather than being consumed in a single operation.

The primary motivation is therefore to explore whether proxy-signature-based delegation can provide a lightweight, account-independent alternative for policy-controlled spending, while making blockchain-based applications more accessible to users who should not have to manage the underlying wallet infrastructure.

The Problem of Consumption

When examining anonymous cryptocurrencies and token allowance protocols, we can identify an interesting limitation shared by many constructions: the consumption of delegated or issued authorization is often indivisible or restricted to one-time use.

For example, in anonymous cryptocurrency systems, a *nullifier* is typically used to prevent a deposited note or coin from being spent more than once. Once the corresponding nullifier is published, the associated spending authorization is considered consumed. Similarly, one-time authorization mechanisms, such as signatures used in *Permit2*, can be designed to prevent replay by ensuring that an authorization cannot be reused.

However, this behavior is closely tied to the objectives and security requirements of each protocol. Anonymous cryptocurrencies primarily focus on enabling private transfers, deposits, and withdrawals while preventing double-spending. Permit2, on the other hand, focuses on allowing users to authorize smart contracts to access and transfer their tokens under specified conditions.

Although these mechanisms serve their respective purposes effectively, they do not necessarily provide a general mechanism for *partially consuming delegated authority while preserving the remaining authorization for future use.*

Objective

Partial consumption and transparency are the fundamental principles of this protocol. The term *spending* is not limited to monetary transactions but encompasses any resource or entitlement that can be consumed incrementally.

The protocol leverages decentralized infrastructure for cryptographic verification and state management, while relying on a central authority to issue and manage allowances. This hybrid architecture combines decentralized enforcement and verifiability with centralized allowance administration.

High level Idea

Suppose Alice deposits 10 ETH into protocol contract. She defines three independent spending policies on behalf of Bob, John, and Smith:

- Bob is authorized to spend up to 4 ETH.
- John is authorized to spend up to 4 ETH.
- Smith is authorized to spend up to 2 ETH.

Alice generates three distinct secret tokens and distributes one to each of them, allowing them to consume their respective allowances however they wish.

Revealing or consuming a secret token would be straightforward if the entire allowance were spent in a single transaction. However, DSP adopts the concept of *proxy signatures*, allowing each authorization to be exercised through multiple transactions.

Consider Bob, who owns two Ethereum accounts. He decides to transfer 1 ETH to his secondary account and 3 ETH to his primary account. He can do so because the authorization is associated with his secret token and spending policy, rather than being restricted to a particular Ethereum account.

John, meanwhile, withdraws only 3 ETH from his 4 ETH allowance, leaving 1 ETH unconsumed.

Unfortunately, Smith loses his secret token before using it. Since the token represents his delegated spending authority, he contacts Alice and requests a replacement. Alice revokes the previous authorization and prepares a new token for Smith, allowing him to continue under the same spending policy.

Smith is happy to receive his replacement and eventually withdraws his full 2 ETH allowance.

After some days, John decides not to use his remaining 1 ETH. Since this amount was never consumed, it remains available to Alice under the protocol's withdrawal policy. Alice can therefore reclaim the remaining 1 ETH.

Ultimately, Bob consumes 4 ETH, Smith consumes 2 ETH, John consumes 3 ETH, and Alice reclaims the remaining 1 ETH.

Adoption in Next-Generation Digital Wallets

Another potential application of the Delegated Spending Protocol (DSP) is the development of next-generation digital wallets. At first, the idea of building a digital wallet on top of an existing wallet may sound unusual. However, the proposed architecture follows a concept similar to the integration of traditional e-banking services

with digital wallets, where users load funds into a wallet and subsequently spend them through its interface.

DSP introduces a different approach to this integration. Instead of transferring funds into the digital wallet, the spender generates a cryptographic spending token and transfers it to the digital wallet. This token represents a predefined spending authority, allowing the digital wallet to initiate payments on behalf of the spender without necessarily taking custody of the underlying funds.

When a payment is initiated, the digital wallet presents the token and the corresponding authorization to the underlying spending mechanism. The protocol verifies the authorization, checks the applicable spending policy and remaining allowance, and permits the transaction only if the conditions are satisfied.

This approach separates the custody of funds from the authority to spend them. The user's original wallet retains control over the funds, while the digital wallet operates as an interface through which delegated spending permissions can be exercised.

Consequently, DSP could enable digital wallets to facilitate payments without requiring users to deposit or transfer their funds into the digital wallet beforehand, Rather authorize transfer of balance to other wallet address.

It solves the following:

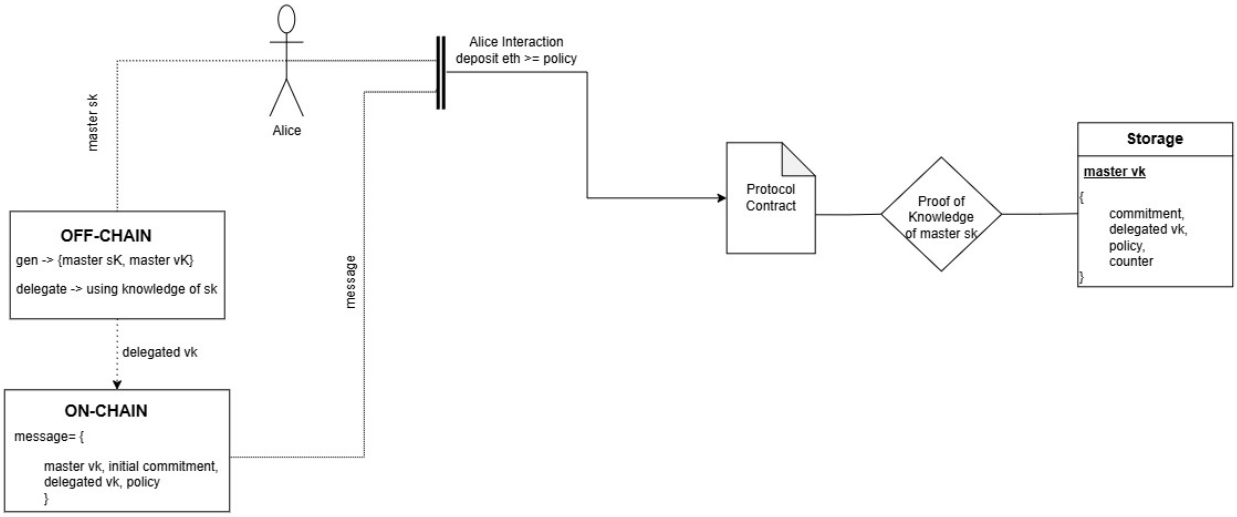
- Non-technical users: They don't need to understand blockchain transactions, gas fees, private keys, or wallet interfaces. They interact with a familiar digital wallet.

- Technical users: They don't need to unlock their primary wallet and manually authorize every payment. They can delegate a bounded spending allowance to their digital wallet, Reducing private-key exposure.

Similarly there are numerous application such as allocation of budget, subscription based protocol, parental control spending. Automated agents -: An AI agent or automated service receives permission to pay for APIs computing resource or service upto a defined budget.

How It Works

Policy Setup



Initial setup by Alice for defining spending and consuming policy

The most important aspect of the protocol is the ability of Alice (the resource owner) to define an appropriate policy governing how consumers can utilize the allocated resources.

The initialization process begins with the generation of a cryptographically secure key pair consisting of a private key and its corresponding public key. These keys must be independent of Alice's actual wallet key pair and are used exclusively within the protocol. We refer to them as the *Master Secret Key* (*Master SK*) and the *Master Verification Key* (*Master VK*), respectively.

Using the Master SK, Alice generates a predefined number of delegated key pairs, subject to the allocation limits established by the protocol. Each delegated key pair consists of a delegated secret key (*Delegated SK*) and its corresponding delegated verification key (*Delegated VK*). The delegated secret keys are securely distributed to their respective consumers through an off-chain communication channel, while the corresponding verification keys are registered on-chain.

Alice then deposits ETH into the protocol contract along with a cryptographic message defining the delegation and its associated spending restrictions. At a high level, this message contains the following components:

$$M = \left\{ \begin{array}{l} \text{Master VK}, \\ \text{Commitment}, \\ \text{Delegated VK}, \\ \text{Policy} \end{array} \right\}$$

These components are cryptographically bound together and establish the authorization context under which consumers are permitted to spend the deposited resources. The contract uses this information to verify the delegation and enforce the corresponding spending conditions.

Before accepting the delegation, the contract verifies that Alice possesses the secret corresponding to the Master VK. This is achieved through a Sigma protocol that

demonstrates knowledge of the Master SK while establishing the validity and binding of the associated delegation parameters, including the commitment, Delegated VK, and policy.

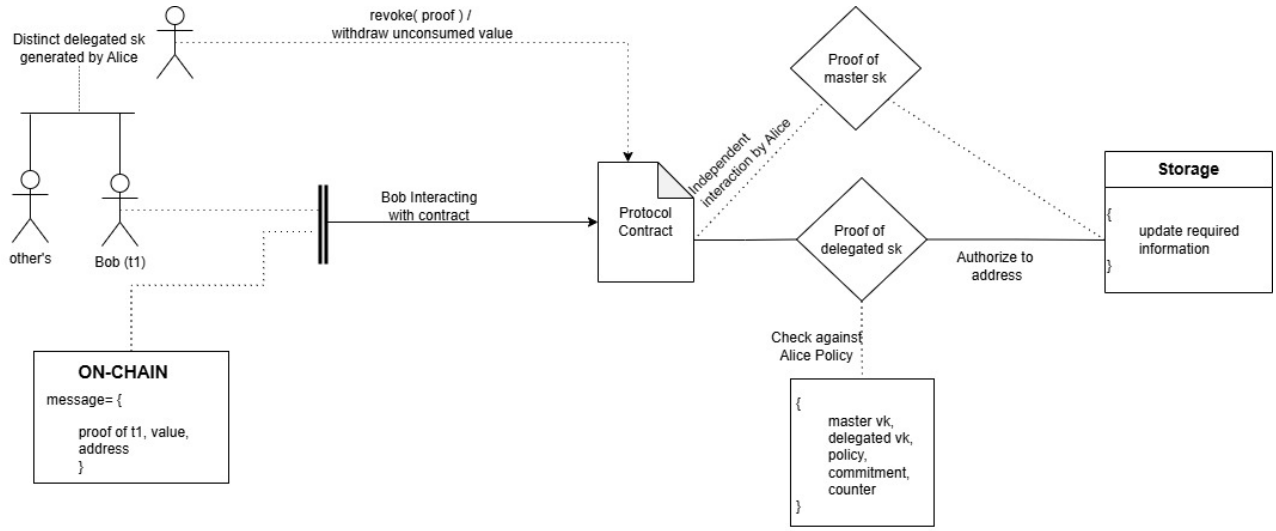
In other words, the proof does not merely establish knowledge of a secret key; it establishes that the submitted delegation parameters are authorized by the owner of the corresponding Master VK.

Once verification succeeds, the contract accepts the delegation and initializes a monotonically increasing counter associated with its authorization state.

The counter plays an important role in preventing replay attacks. Each consumer authorization is bound to the current counter value. Following a successful spending operation, the counter is incremented, invalidating previously submitted authorization messages. Consequently, consumers must generate fresh authorization proofs or signatures bound to the updated counter before exercising their delegated spending rights again.

With the initialization and verification process complete, the delegated consumers can begin exercising their authorization under the restrictions established by Alice.

Consumption



Consumption by Bob who hold's delegated sk

Rather than receiving the inputs required to derive the challenge from an external source, the contract derives them internally. The consumer (Bob) is only required to demonstrate knowledge of the delegated secret key (Delegated SK), while retaining the flexibility to decide how the allocated allowance should be used. For example, Bob may withdraw the funds to an address under his control or specify another address as the recipient by providing it in the transfer request.

A relayer-based model, as it allows a relayer to redeem the allowance on Bob's behalf. In this model, Bob is not required to own or operate a wallet. Instead, he can authorize another party to execute the transfer by providing a valid proof of possession of the Delegated SK. The proof does not merely establish knowledge of a secret key; it establishes that the submitted message is authorized under the corresponding Delegated VK, which is cryptographically bound to the Master VK and the commitment.

Upon successful verification and execution, the counter is incremented, invalidat-

ing previously submitted messages associated with the preceding state. In addition, the contract maintains the amount of resources consumed by Bob and checks each requested expenditure against the constraints defined by the policy. This ensures that Bob cannot exceed the allowance allocated to him, even when making multiple partial withdrawals.

Similarly, Alice, who possesses the Master SK, can revoke the delegation by demonstrating knowledge of the Master SK through a proof bound to the current protocol state, including the counter. Although this mechanism is feasible, its implementation is deferred beyond the initial phase of the protocol. Alice should also be able to withdraw any unconsumed allowance. However, the withdrawal mechanism remains an open design consideration. In particular, I believe that a predefined delay should be imposed before Alice can recover the remaining funds, preventing immediate withdrawal from interfering with an ongoing delegated spending process.

Technology Overview

Proxy signatures are at the core of our implementation. Although the construction follows the general principles of Schnorr signatures, it requires several customizations to accommodate the requirements of our protocol. These include message consistency, separation of on-chain and off-chain computation, and, most importantly, protection against replay attacks.

In conventional one-time signature schemes, a unique signature and its associated nonce are sufficient for authorizing a single resource consumption, provided that the protocol correctly enforces one-time usage. However, our protocol supports incremental and repeated consumption of delegated resources. Consequently, a monotonically increasing counter provides a more suitable replay-protection mechanism. The smart contract maintains this counter and uses its current value to establish the next signing challenge.

The construction will be demonstrated using the BLS12-381 elliptic curve. This choice is motivated by the availability of dedicated Ethereum precompiles for BLS12-381 operations, whereas performing the required native secp256k1 elliptic-curve operations directly within the Ethereum Virtual Machine can be computationally expensive.

This construction serves as the general cryptographic framework for delegated authorization and spending within our protocol. The architecture will be included in future version.

Costs

The protocol is not account-based; instead, authorization relies on proof of possession of a delegated secret key. Consequently, any on-chain observer can associate a transaction with a particular delegation, but cannot determine the real-world identity of the individual possessing or using that delegation. The protocol does not provide transaction confidentiality: all transactions and their associated on-chain data remain publicly observable.

One of the protocol's objectives is to improve the user experience and make blockchain applications accessible to individuals who do not want to deal with wallet connections, private-key management, or the complexities of operating a blockchain account. This can be achieved through a middleware layer whose primary responsibility is to construct and sign transactions, submit them to RPC endpoints, and receive the corresponding transaction fee in return. In this arrangement, the holder of the delegated secret key authorizes the intended operation, while the middleware handles its submission and execution on-chain.

The on-chain component is primarily responsible for verifying the authorization message and transferring resources according to the delegation's defined policy. At

the current design stage, the estimated cryptographic workload for a delegated spending operation consists of two elliptic-curve point additions, one scalar multiplication, and Keccak hashing. The contract must also update the spending counter and the remaining allowance after a successful operation. In addition to these operations, the master secret key holder incurs an initial cost when registering the required on-chain state for each delegation. The protocol does not require pairing-based operations.

These costs are preliminary estimates and must be explicitly evaluated through implementation and gas measurements. Although the overall architecture is described above, some details remain open to refinement. For example, the message representation may be simplified by storing only the ephemeral public key instead of both the commitment and the delegated verification key. The mechanisms for revocation and withdrawal of unconsumed allowances also remain subjects for further design and evaluation.

Future Work

Future work will primarily focus on optimizing on-chain gas consumption to make the protocol more practical and economically feasible for consumers.

Another direction is to explore post-quantum-secure delegation and proxy-signature constructions, where authorization security relies on problems such as Learning With Errors (LWE), or other post-quantum hardness assumptions, rather than the discrete logarithm problem.

These directions remain open for further research and evaluation.